

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 8

Roll no:06 PRN No: 12413462 Name of Student: Purva Patil

Subject Name : Deep Learning

Problem Statement :

Assignment 8 - Implement Pretrained BERT Model for sentiment analysis or text classification on a sample dataset

Algorithm :

1. Import the required Python libraries such as Transformers, PyTorch, Pandas, NumPy, and Scikit-learn.
2. Load the sample sentiment dataset containing text reviews and their corresponding sentiment labels.
3. Explore the dataset by checking the number of samples, text data, and sentiment classes.
4. Split the dataset into training and testing sets.
5. Load the pretrained BERT tokenizer using the bert-base-uncased model.
6. Tokenize the text data using the BERT tokenizer by converting sentences into token IDs, attention masks, and other required inputs.
7. Create a dataset object compatible with PyTorch for training and testing.
8. Load the pretrained BERT model using BertForSequenceClassification and set the number of output classes to 2 for positive and negative sentiment.
9. Define the evaluation metric using classification accuracy.
10. Set the training parameters such as number of epochs, batch size, learning rate, and weight decay.
11. Fine-tune the pretrained BERT model using the training dataset.
12. Evaluate the trained model on the testing dataset and calculate its accuracy.
13. Generate predictions for new unseen sentences using the fine-tuned BERT model.
14. Display the predicted sentiment as Positive or Negative.
15. Analyze the results using accuracy and classification metrics and conclude the performance of the BERT model.

Code And Output :

```
!pip install -q transformers datasets accelerate scikit-learn
```

```
import pandas as pd
import numpy as np
import torch

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    Trainer,
    TrainingArguments
)
```

```
data = {
    "text": [
        "I loved this movie, it was amazing!",
        "The product quality is excellent.",
        "I really enjoyed the experience.",
        "This is the best service I have ever received.",
        "The food was delicious and fresh.",
        "I am very happy with my purchase.",
        "The movie was fantastic and entertaining.",
        "The service was quick and friendly.",
        "I hated this movie, it was terrible.",
        "The product quality is very poor.",
        "I did not enjoy the experience.",
        "This is the worst service ever.",
        "The food was cold and tasteless.",
        "I am disappointed with my purchase.",
        "The movie was boring and terrible.",
        "The service was slow and rude."
    ],
    "label": [
        1, 1, 1, 1, 1, 1, 1, 1, 1,
        0, 0, 0, 0, 0, 0, 0, 0, 0
    ]
}

df = pd.DataFrame(data)

df
```

	text	label
0	I loved this movie, it was amazing!	1
1	The product quality is excellent.	1
2	I really enjoyed the experience.	1
3	This is the best service I have ever received.	1
4	The food was delicious and fresh.	1
5	I am very happy with my purchase.	1
6	The movie was fantastic and entertaining.	1
7	The service was quick and friendly.	1
8	I hated this movie, it was terrible.	0
9	The product quality is very poor.	0
10	I did not enjoy the experience.	0
11	This is the worst service ever.	0
12	The food was cold and tasteless.	0
13	I am disappointed with my purchase.	0
14	The movie was boring and terrible.	0
15	The service was slow and rude.	0

```
train_texts, test_texts, train_labels, test_labels = train_test_split(
    df["text"],
    df["label"],
    test_size=0.25,
    random_state=42,
    stratify=df["label"]
)

print("Training samples:", len(train_texts))
print("Testing samples:", len(test_texts))
```

```
model_name = "bert-base-uncased"

tokenizer = BertTokenizer.from_pretrained(model_name)

train_encodings = tokenizer(
    list(train_texts),
    truncation=True,
    padding=True,
```

```
        max_length=128
    )

test_encodings = tokenizer(
    list(test_texts),
    truncation=True,
    padding=True,
    max_length=128
)

print("Tokenization completed!")
```

```
class SentimentDataset(torch.utils.data.Dataset):

    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = list(labels)

    def __getitem__(self, idx):
        item = {
            key: torch.tensor(value[idx])
            for key, value in self.encodings.items()
        }
        item["labels"] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)

train_dataset = SentimentDataset(
    train_encodings,
    train_labels
)

test_dataset = SentimentDataset(
    test_encodings,
    test_labels
)

print("Dataset ready!")
```

```
model = BertForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)

print("BERT model loaded successfully!")
```

config.json: 100%

570/570 [00:00<00:00, 11.0kB/s]

model.safetensors: reconstructing file: 100%

440MB / 440MB, 38.4MB/s

model.safetensors: downloading bytes:

415MB, 34.7MB/s

Loading weights: 100%

199/199 [00:00<00:00, 4645.58it/s]

[transformers] **BertForSequenceClassification** **LOAD REPORT** from: bert-base-uncased

Key	Status
-----	--------

cls.seq_relationship.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

BERT model loaded successfully!

```
def compute_metrics(eval_pred):  
  
    logits, labels = eval_pred  
  
    predictions = np.argmax(logits, axis=-1)  
  
    accuracy = accuracy_score(labels, predictions)  
  
    return {  
        "accuracy": accuracy  
    }
```

```

training_args = TrainingArguments(
    output_dir="./bert_results",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    learning_rate=2e-5,
    weight_decay=0.01,
    logging_steps=5,
    report_to="none"
)

```

```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)

```

```

trainer.train()

```

```

/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning:
'pin_memory' argument is set as true but no accelerator is found, then device pinned
memory won't be used.

```

```

    super().__init__(loader)

```

```

[9/9 00:29, Epoch 3/3]

```

Step Training Loss

```

5          0.686704

```

```

Writing model shards: 100%

```

```

1/1 [00:03<00:00, 3.44s/it]

```

```

TrainOutput(global_step=9, training_loss=0.6786449220445421, metrics={'train_runtime':
33.6778, 'train_samples_per_second': 1.069, 'train_steps_per_second': 0.267,
'total_flos': 221999952960.0, 'train_loss': 0.6786449220445421, 'epoch': 3.0})

```

```

results = trainer.evaluate()

```

```

print("Evaluation Results:")

```

```

print(results)

```

```

/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning:
'pin_memory' argument is set as true but no accelerator is found, then device pinned
memory won't be used.

```

```

    super().__init__(loader)

```

```

[1/1 :<:]

```

Training Loss Validation Loss Step Accuracy

0.686704 0.628994 9 0.750000

Evaluation Results:

```
{'eval_loss': 0.6289941668510437, 'eval_accuracy': 0.75}
```

```
predictions = trainer.predict(test_dataset)
```

```
predicted_labels = np.argmax(
    predictions.predictions,
    axis=1
)
```

```
print(classification_report(
    test_labels,
    predicted_labels,
    target_names=["Negative", "Positive"]
))
```

/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.

```
super().__init__(loader)

      precision    recall  f1-score   support

   Negative       0.67      1.00      0.80         2
   Positive       1.00      0.50      0.67         2

   accuracy              0.75         4
  macro avg       0.83      0.75      0.73         4
 weighted avg       0.83      0.75      0.73         4
```

```
def predict_sentiment(text):
```

```
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=128
    )
```

```
    device = model.device
    inputs = {
        key: value.to(device)
        for key, value in inputs.items()
    }
```

```
    with torch.no_grad():
        outputs = model(**inputs)
```

```
prediction = torch.argmax(
    outputs.logits,
    dim=1
).item()

if prediction == 1:
    return "Positive"
else:
    return "Negative"
```

```
def predict_sentiment(text):

    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=128
    )

    device = model.device
    inputs = {
        key: value.to(device)
        for key, value in inputs.items()
    }

    with torch.no_grad():
        outputs = model(**inputs)

    prediction = torch.argmax(
        outputs.logits,
        dim=1
    ).item()

    if prediction == 1:
        return "Positive"
    else:
        return "Negative"
```

```
sentences = [
    "I am very disappointed.",
    "This movie was fantastic."
]

for sentence in sentences:
    print(sentence)
```

```
print("Sentiment:", predict_sentiment(sentence))  
print()
```

```
I am very disappointed.  
Sentiment: Negative
```

```
This movie was fantastic.  
Sentiment: Positive
```